

ХАКАТОН · ЗАДАЧА 2

Умный роутинг выплат

Движок распределения выплат между платёжными провайдерами:
допуск, выбор, каскад, объяснение и рекомендации

Команда «Коломот» · Фёдор Филитович · Артём Корсаков

Ruby 3.3 · 0 внешних зависимостей · 122 теста · github.com/T0vich/kolomot-smart-payout-routing

«Выбрать подходящего партнёра» — это три разных вопроса

01

Кому **можно**?

Ответ бинарный. Лимиты, банки, реквизиты, маржа. Нарушать нельзя никогда — независимо от того, какая стратегия включена.

02

Кого **лучше**?

Ответ — всегда компромисс. Семь целей из ТЗ конфликтуют между собой, и это не баг постановки, а её суть.

03

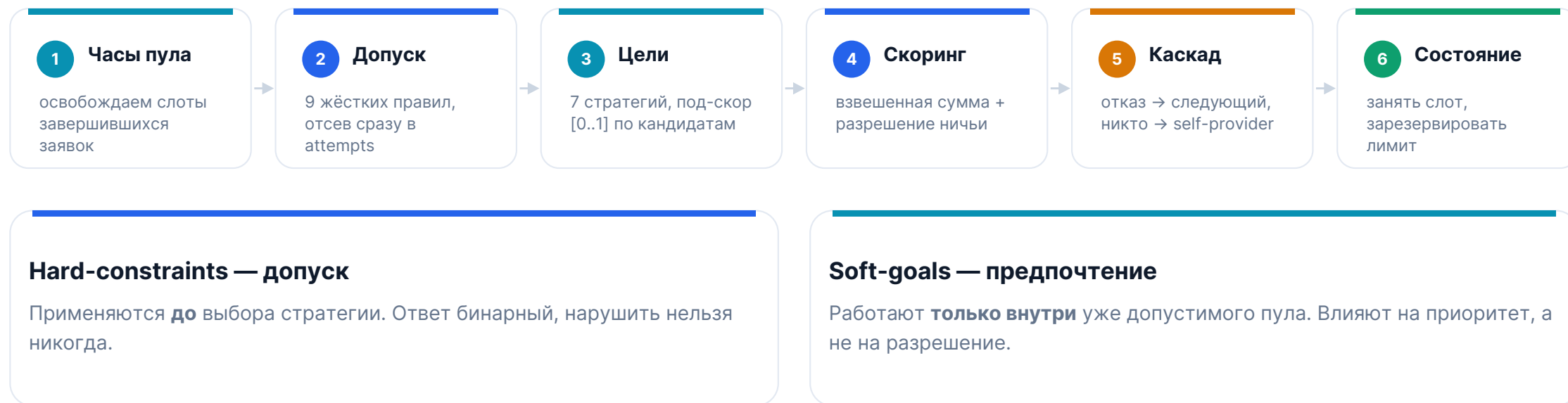
Что если **отказал**?

Нужна последовательность попыток, которая ни на одном шаге не нарушает жёсткие ограничения.

Пример конфликта прямо из ТЗ. Цель — 40 % заявок на вірау. Факт — вірау почти исчерпал дневной максимум, а на payflow висит обязательство «не менее 2 млн ₴ в сутки». Обе цели корректны. Одновременно выполнить нельзя. Кому отдаём заявку — и как это объяснить?

Большинство ошибок в таких системах — от попытки решить все три вопроса одним механизмом.

Два слоя, которые нельзя смешивать



Стратегия физически не может «угговорить» роутер нарушить лимит. Альтернативу — единый скоринг со штрафом за нарушение — мы отвергли: там нарушение перестаёт быть невозможным и становится просто дорогим.

9 жёстких ограничений — все из ТЗ, по классу на правило

Статус провайдера

provider_inactive

Диапазон суммы чека

amount_below_minimum · amount_exceeds_limit

Дневной максимум

daily_limit_exceeded

Одновременные заявки

in_progress_count_exceeded ·
in_progress_amount_exceeded

Банковский фильтр

bank_not_in_list · bank_in_exclude_list

Маржа

negative_margin

Реквизиты

no_requisites

Интенсивность

rate_limit_exceeded

Потолок оборота — наше

turnover_max_reached

```
{ "provider": "payflow", "decision": "skipped", "stage": "hard_filter",  
  "reason": "amount_exceeds_limit", "details": "150000 > limit_amount_max 50000" }
```

Отсев всегда объяснён числами. Состав и порядок правил задаётся **config/routing.yml** — любое выключается без правки кода.

Семь стратегий распределения — все семь из ТЗ

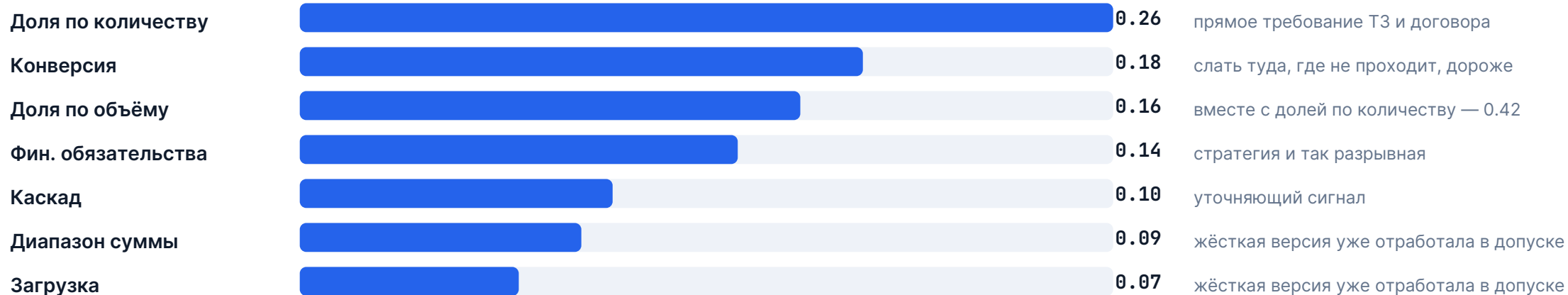
#	Стратегия	Поле провайдера	Что делает
1	Доля по количеству	traffic_percentage	Минимизирует суммарное отклонение всех фактических долей от целевых
2	Доля по объёму	volume_share_pct	То же самое, но доли считаются в рублях
3	Очередь в каскаде	priority	Ранг считается среди доступных , а не по всему списку
4	По сумме чека	amount_bands	Диапазон влияет на предпочтение , а не только отсекает
5	По конверсии	conversion_24h	Заявленная, смешанная с фактической по истории
6	По интенсивности	requests_per_minute_limit	Текущая загрузка влияет на выбор , а не только проверяется потолок
7	Фин. обязательства	daily_turnover_min / max	Дотягивает минимальный оборот, притормаживает у максимума

Каждая — отдельный класс, каждая нормирует свой сигнал в [0..1], каждая включается весом в профиле. Вес 0 — стратегия в прогоне не участвует.

Все семь целей сразу — взвешенный композитный балл

$$\text{score(провайдер)} = \sum (\text{под-скор}_i \times \text{вес}_i) / \sum \text{вес}_i$$

Веса профиля `balanced`, сдаваемого по умолчанию:



Веса живут в `config/routing.yml`, а не в коде: завтра приоритетом станет конверсия — меняется одна строка. Профиль `conversion_first` уже в репозитории и включается флагом `--profile`.

Что делаем, когда цели указывают на разных провайдеров

01

Разрыв баллов мал

Применяется явный порядок политик **conflict_order**:

фин. обязательства → доля по количеству → конверсия → доля по объёму → каскад → диапазон суммы → загрузка

02

Цели равны полностью

Детерминированный разрыв: сначала **priority**, затем имя провайдера.

Один и тот же вход всегда даёт один и тот же выход — это отдельный тест.

03

Цель невыполнима

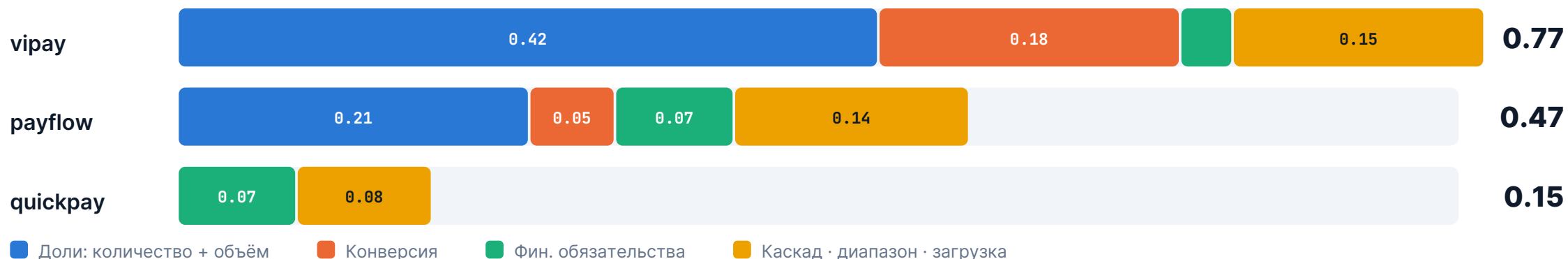
Требуемая доля относится к провайдеру, которого отсекают hard-constraints.

Цель **не блокирует** роутинг: заявка уходит дальше, расхождение — в отчёт с рекомендацией.

Почему обязательства выше долей. За недобор минимального оборота платят деньгами по договору. Целевая доля — ориентир, её можно добрать следующими заявками. Порядок задан конфигом и проверяется при старте: неизвестная стратегия в **conflict_order** — ошибка запуска, а не тихое игнорирование.

Не «выбрали vipay», а почему он и почему не остальные

op_101 · 15 000 ₽ · Сбербанк. Допустимы все трое, и цели расходятся: по диапазону суммы лучше payflow, по загрузке — quickpay, по долям и конверсии — vipay.



В **attempts** у выбранного — балл, разложение по слагаемым и текст с ближайшим конкурентом. У остальных — конкретная причина: **amount_exceeds_limit**, **bank_not_in_list**, **lower_score** — и расшифровка числами.

Различаем два события, названные в ТЗ одним словом

Провайдер не принял заявку

Ретраибельный отказ, заявка в работу не ушла.

Исключаем из пула → следующий кандидат. Кандидаты кончились → self-provider.

Провайдер принял и обработал

Терминальный исход: approved / rejected / expired.

Маршрут не меняется, исход пишется в simulated_result.

Каскад в действии — профиль demo_failover, заявка op_103 на 150 000 ₽:

```
vipay      skipped  amount_exceeds_limit  150000 > limit_amount_max 100000
payflow    skipped  amount_exceeds_limit  150000 > limit_amount_max 50000
quickpay   skipped  provider_declined     провайдер отказался принять заявку
spacepayments selected fallback_self_provider внешних допустимых не осталось
```

В сдаваемом профиле искусственные отказы выключены осознанно: эталон организаторов требует для op_103 строго quickpay, а он там единственный допустимый — случайный отказ увёл бы заявку в spacepayments и провалил проверку. Механика не исчезла: **rake demo** и четыре теста.

Лимиты резервируем, а не считаем пост-фактум

Наивная реализация

`daily_approved_amount` растёт только после подтверждения выплаты.

Десять заявок, отправленных подряд, все проходят проверку дневного лимита — и лимит продан десять раз.

Наша реализация

Сумма резервируется в момент выбора провайдера:

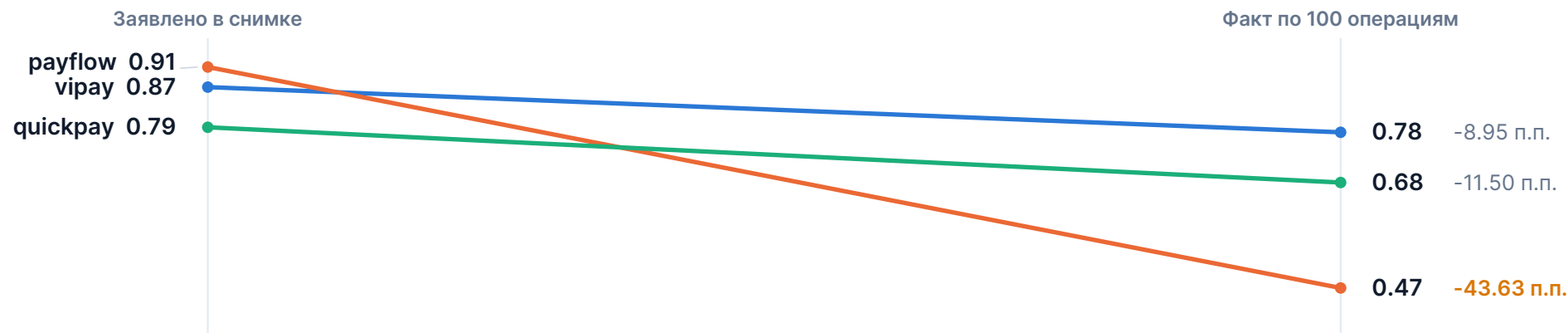
`занято = daily_approved_amount + daily_reserved_amount`

При завершении резерв снимается. То же для in-progress count и amount.

Побочная выгода. Валидатор организаторов считает лимиты по исходному снимку `providers.json`. Наша проверка строго жёстче — значит, выбранный нами провайдер гарантированно входит в его список допустимых. Результат: **29 проверок пройдено, 0 ошибок, 0 предупреждений.**

Плюс модельное время: заявка держит слот `latency_sec` и освобождает его. На публичной очереди из 10 заявок разницы не видно, на длинной тестовой — решает, упрётся провайдер в лимит одновременных заявок или нет.

Отчёт называет причину расхождения и параметр к изменению



Главная находка по данным. Провайдер с самой высокой заявленной конверсией на деле худший: payflow обещает 0.91, по 100 историческим операциям даёт 0.47. Роутинг, который берёт conversion_24h как есть, будет систематически слать заявки туда, где они не проходят. Поэтому у нас конверсия смешанная: **history_blend = 0.3**.

Рекомендации называют провайдера, число и параметр: «payflow выбрал 98.29 % дневного лимита – поднять daily_amount_limit до 3 600 000 ₽ или снизить его долю трафика».

Переиграли 100 исторических операций через наш роутер



32 → 26 п.п.

суммарное отклонение от целевых долей: было в истории → стало у нас

29 из 100

исторических выплат ушли провайдеру, который по правилам допуска не мог их принять

26 vs 38 п.п.

balanced против conversion_first при конверсии 0.66 и 0.69 — цена политики

Метрика намеренно не замкнута на себя: ожидаемая конверсия считается по фактической успешности провайдеров из той же истории, а не по нашему симулятору.

Что нужно изменить — и сколько для этого строк кода

Задача	Что делаем	Строк кода
Добавить провайдера	Строка в providers.json	0
Изменить лимиты, банки, конверсию, целевые доли	Правка providers.json	0
Изменить приоритеты бизнеса	Вес профиля в config/routing.yml	0
Изменить порядок разрешения конфликтов	Ключ conflict_order в конфиге	0
Выключить любое правило допуска	Ключ в разделе constraints конфига	0
Другая политика роутинга целиком	Новый профиль + флаг --profile	0
Новое правило допуска	Класс в constraints/ + строка в реестре + ключ в конфиге	~25
Новая стратегия	Класс в strategies/ + строка в реестре + вес в профиле	~30

Ни одно имя провайдера не зашито в код. Семь профилей уже в репозитории:

- balanced
- cascade
- traffic
- volume
- conversion_first
- commitments
- demo_failover

Что сделано и как это проверить

122

теста, 796 проверок

29 / 29

проверок валидатора

0

внешних зависимостей

7

профилей роутинга

CI

зелёный на каждый push

Тесты без единой внешней зависимости

`rake test`

Прогон публичной очереди

`rake route`

Валидатор организаторов

`rake validate`

Каскад и fallback в действии

`rake demo`

Разбор истории и сравнение политик

`rake analyze · compare`

Сдаваемые файлы из тестовой очереди

`rake submit`

Готовы к вопросам

Ограничения кейса соблюдены: нейросетей внутри решения нет, проприетарных компонентов нет, весь код — Ruby на стандартной библиотеке. Повторный запуск на тех же данных даёт побайтово тот же результат.

Почему веса именно такие

- Веса — не подобранная магия, а расстановка приоритетов, которую можно защитить словами. **Доли по количеству и объёму вместе дают 0.42** — это прямое требование ТЗ и договора.
- **Конверсия 0.18** — второй по величине одиночный вес: отправлять туда, где выплата не проходит, дороже, чем немного отклониться от доли.
- **Фин. обязательства 0.14** — вес умеренный, потому что стратегия и так разрывная: она выстреливает только когда минимум не набран.
- **Каскад, диапазон суммы, загрузка — 0.26 на троих**: это уточняющие сигналы, жёсткие версии этих же ограничений уже отработали в слое допуска.
- Проверка на истории: профили **traffic, volume, commitments** и **balanced** дают одинаковые 26 п.п. Результат устойчив к перекосу весов, а не держится на одной подобранной комбинации.

Нормировка: сравнительные и абсолютные сигналы

Сравнительные — min-max по пулу

traffic_share, volume_share, conversion, cascade_priority.

Разброс конверсий в кейсе — 0.79...0.91. Без нормировки эта разница растворяется среди остальных слагаемых и не влияет ни на что.

Абсолютные — по своей шкале

load_balance, turnover_commitment, amount_band.

«Загружен на 64 %» осмысленно само по себе: сравнивать не с чем и не нужно.

Следствие, о котором стоит сказать честно. При двух кандидатах сравнительные стратегии дают ровно 1.0 и 0.0, и выбор превращается во взвешенное голосование. На публичной очереди из 10 заявок это видно: 4 заявки безальтернативны, на остальных шести все содержательные профили сходятся к одному решению. Расхождение политик проявляется на объёме — поэтому у нас есть backtest.

Обработка ошибок и граничных ситуаций

Ситуация	Поведение
Файл не найден, битый JSON	Ошибка с путём и причиной разбора, а не стектрейс
amount строкой, нулём, отрицательным; заявка без id	Отклоняется на загрузке с указанием поля и значения
Дубли operation_id	Отклоняются: молча смаршрутизировать дубль хуже, чем упасть
Очередь — один объект вместо массива	Принимается: валидатор организаторов допускает оба варианта
Пустая очередь	Валидный отчёт с нулями, без деления на ноль
providers.json не массив / без обязательного поля	Ошибка с указанием, чего именно не хватает
Неизвестная стратегия или правило в конфиге	Ошибка при старте, а не тихое игнорирование
Под заявку не подходит вообще никто	Уходит на spacepayments, помечается fallback_used
daily_amount_limit = null	Отсутствие лимита, а не ноль
У провайдера кончились реквизиты посреди очереди	Покидает пул, остальные заявки идут дальше

Всё перечисленное закрыто тестами: 11 в loaders_test.rb, 9 в config_test.rb, 18 в report_test.rb, 17 в router_test.rb.

Спорные места ТЗ: что выбрали и что переключается

Вопрос	Наша трактовка	Если прочтение другое
Неуспешная выплата — отказ, требующий перемаршрутизации, или финал?	Финал. К следующему переходим, только если провайдер не принял заявку в работу	Флаг <code>retry_on_terminal_failure</code> — реализован, покрыт тестами, по умолчанию выключен
Скрытая проверка считает лимиты по исходному снимку <code>providers.json</code> ?	Да — поэтому мы намеренно строже: резервируем лимит в момент выбора	Ослабить резервирование в конфиге; менее строгий вариант нам не нужен
Можно ли добавлять свои поля в решение?	Да, ТЗ прямо разрешает дополнительные поля	Убрать <code>score</code> , <code>score_breakdown</code> , <code>stage</code> , <code>eligible_providers</code> из <code>Decision#to_h</code>
Дополнительные поля провайдера — в конфиг или в <code>providers.json</code> ?	Держим в конфиге, но код читает оба источника	Ничего менять не нужно

Отдельной QA-сессии с ответами организаторов не было, поэтому по каждому спорному месту мы выбрали более строгий вариант, записали обоснование в **docs/QA-ANSWERS.md** и заранее подготовили переключатель.